

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Dessy Nurulita

NIM. 2310817220024

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Dessy Nurulita
NIM : 2310817220024

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	6
B. Output Program	25
C. Pembahasan	30
D. Tautan Git	48

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 (Home Light Mode).....	25
Gambar 2. Screenshot Hasil Jawaban Soal 1 (Detail Light Mode).....	26
Gambar 3. Screenshot Hasil Jawaban Soal 1 (Home Dark Mode).....	27
Gambar 4. Screenshot Hasil Jawaban Soal 1 (Detail Dark Mode).....	28
Gambar 5. Screenshot Hasil Jawaban Soal 1 (Website).....	29

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1 (ConstellationDao.kt).....	6
Tabel 2. Source Code Jawaban Soal 1 (ConstellationDatabase.kt).....	7
Tabel 3. Source Code Jawaban Soal 1 (ConstellationEntity.kt).....	7
Tabel 4. Source Code Jawaban Soal 1 (ApiResponse.kt).....	8
Tabel 5. Source Code Jawaban Soal 1 (Constellation.kt)	8
Tabel 6. Source Code Jawaban Soal 1 (ApiService.kt).....	8
Tabel 7. Source Code Jawaban Soal 1 (RetrofitInstance.kt)	9
Tabel 8. Source Code Jawaban Soal 1 (ConstellationRepository.kt).....	10
Tabel 9. Source Code Jawaban Soal 1 (ListConstellationAdapter.kt)	10
Tabel 10. Source Code Jawaban Soal 1 (DetailFragment.kt).....	12
Tabel 11. Source Code Jawaban Soal 1 (HomeFragment.kt).....	13
Tabel 12. Source Code Jawaban Soal 1 (MainActivity.kt)	14
Tabel 13. Source Code Jawaban Soal 1 (Resource.kt)	16
Tabel 14. Source Code Jawaban Soal 1 (MainViewModel.kt)	16
Tabel 15. Source Code Jawaban Soal 1 (MainViewModelFactory.kt)	17
Tabel 16. Source Code Jawaban Soal 1 (AndroidAPIApp.kt)	18
Tabel 17. Source Code Jawaban Soal 1 (activity_main.xml).....	18
Tabel 18. Source Code Jawaban Soal 1 (fragment_detail.xml)	19
Tabel 19. Source Code Jawaban Soal 1 (fragment_home.xml)	20
Tabel 20. Source Code Jawaban Soal 1 (star_item.xml).....	20
Tabel 21. Source Code Jawaban Soal 1 (menu_theme.xml)	23
Tabel 22. Source Code Jawaban Soal 1 (navigation_graph.xml).....	23
Tabel 23. Source Code Jawaban Soal 1 (themes.xml)	24
Tabel 24. Source Code Jawaban Soal 1 (themes.xml (night)).....	24

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- Gunakan caching strategy pada Room
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. data/local/ConstellationDao.kt

Tabel 1. Source Code Jawaban Soal 1 (ConstellationDao.kt)

1	package	com.example.modul5.data.local
2		
3	import	androidx.room.*
4	import	kotlinx.coroutines.flow.Flow
5		
6	@Dao	
7	interface	ConstellationDao {
8	@Query("SELECT * FROM constellations")	
9	fun getAll() :	Flow<List<ConstellationEntity>>
10		
11	@Insert(onConflict = OnConflictStrategy.REPLACE)	
12	suspend fun	insertAll(data :

13	List<ConstellationEntity>) }
----	---------------------------------

2. data/local/ConstellationDatabase.kt

Tabel 2. Source Code Jawaban Soal 1 (ConstellationDatabase.kt)

1	package	com.example.modul5.data.local
2		
3	import	android.content.Context
4	import	androidx.room.Database
5	import	androidx.room.Room
6	import	androidx.room.RoomDatabase
7		
8	@Database(entities	= [ConstellationEntity::class],
9	version	= 1, exportSchema = false)
10	abstract class ConstellationDatabase :	RoomDatabase() {
11	abstract fun constellationDao():	ConstellationDao
12		
13	companion	object {
14	@Volatile	
15	private var INSTANCE: ConstellationDatabase?	=
16	null	
17		
18	fun getInstance(context: Context):	ConstellationDatabase
19		{
20	return INSTANCE ?: synchronized(this) {	
21	Room.databaseBuilder(	
22	context.applicationContext,	
23	ConstellationDatabase::class.java,	
24	"constellation_db"	
25	).build().also { INSTANCE = it }	
26	}	
	}	
	}	
	}	

3. data/local/ConstellationEntity.kt

Tabel 3. Source Code Jawaban Soal 1 (ConstellationEntity.kt)

1	package	com.example.modul5.data.local
2		
3	import	android.os.Parcelable
4	import	androidx.room.Entity
5	import	androidx.room.PrimaryKey
6	import	kotlinx.parcelize.Parcelize
7		
8	@Parcelize	
9	@Entity(tableName	= "constellations")
10	data class	ConstellationEntity(

11	@PrimaryKey	val	name:	String,
12	val	description:		String,
13	val	year:		String,
14	val	imageUrl:		String,
15	val	webUrl:		String
16	)	: Parcelable		

4. data/model/ApiResponse.kt

Tabel 4. Source Code Jawaban Soal 1 (ApiResponse.kt)

1	package		com.example.modul5.data.model
2			
3	import		kotlinx.serialization.Serializable
4			
5	@Serializable		
6	data	class	ApiResponse<T> (
7	val	status:	String,
8	val	message:	String,
9	val	data:	T
10	)		
11			

5. data/model/Constellation.kt

Tabel 5. Source Code Jawaban Soal 1 (Constellation.kt)

1	package		com.example.modul5.data.model
2			
3	import		kotlinx.serialization.SerialName
4	import		kotlinx.serialization.Serializable
5			
6	@Serializable		
7	data	class	Constellation (
8	val	name:	String,
9	val	description:	String,
10	val	year:	String,
11	@SerialName("image_url")	val	imageUrl: String,
12	@SerialName("web_url")	val	webUrl: String
13	)		

6. data/remote/ApiService.kt

Tabel 6. Source Code Jawaban Soal 1 (ApiService.kt)

1	package		com.example.modul5.data.local
2	package		com.example.modul5.data.remote
3			
4	import		com.example.modul5.data.model.ApiResponse
5	import		com.example.modul5.data.model.Constellation
6	import		retrofit2.http.GET

7	
8	interface ApiService {
9	@GET("constellation")
10	suspend fun getConstellations(): ApiResponse<List<Constellation>>
	}

7. data/remote/RetrofitInstance.kt

Tabel 7. Source Code Jawaban Soal 1 (RetrofitInstance.kt)

1	package com.example.modul5.data.remote
2	
3	import
4	com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
5	
6	import kotlinx.serialization.json.Json
7	import okhttp3.MediaType.Companion.toMediaType
8	import okhttp3.OkHttpClient
9	import okhttp3.logging.HttpLoggingInterceptor
10	import retrofit2.Retrofit
11	
12	object RetrofitInstance {
13	
14	private const val BASE_URL = "https://constellation-
15	api.free.beeceptor.com/"
16	
17	private val json = Json {
18	ignoreUnknownKeys = true
19	}
20	
21	private val client = OkHttpClient.Builder()
22	.addInterceptor(HttpLoggingInterceptor().apply
23	{
24	level = HttpLoggingInterceptor.Level.BODY
25	})
26	.build()
27	
28	val api: ApiService by lazy {
29	Retrofit.Builder()
30	.baseUrl(BASE_URL)
31	
32	.addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
	.client(client)
	.build()
	.create(ApiService::class.java)
	}
	}

8. data/repository/ConstellationRepository.kt

Tabel 8. Source Code Jawaban Soal 1 (ConstellationRepository.kt)

```
1 package com.example.modul5.data.repository
2
3 import com.example.modul5.data.local.ConstellationDao
4 import
5 com.example.modul5.data.local.ConstellationEntity
6 import com.example.modul5.data.remote.RetrofitInstance
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.flow.flow
9 import kotlinx.coroutines.flow.emitAll
10
11 class ConstellationRepository(
12     private val dao: ConstellationDao
13 ) {
14
15     fun getConstellations():
16     Flow<List<ConstellationEntity>> = flow {
17         try {
18             val response =
19             RetrofitInstance.api.getConstellations()
20             if (response.status == "success") {
21                 val mapped = response.data.map {
22                     ConstellationEntity(
23                         name = it.name,
24                         description = it.description,
25                         year = it.year,
26                         imageUrl = it.imageUrl,
27                         webUrl = it.webUrl
28                     )
29                 }
30                 dao.insertAll(mapped)
31             }
32             catch (e: Exception) {
33                 e.printStackTrace()
34             }
35             emitAll(dao.getAll())
36         }
37     }
```

9. ui/adapter/ListConstellationAdapter.kt

Tabel 9. Source Code Jawaban Soal 1 (ListConstellationAdapter.kt)

```
1 package com.example.modul5.ui.adapter
2
3 import android.util.TypedValue
4 import android.view.LayoutInflater
5 import android.view.ViewGroup
```

```

6  import androidx.recyclerview.widget.DiffUtil
7  import androidx.recyclerview.widget.ListAdapter
8  import androidx.recyclerview.widget.RecyclerView
9  import com.bumptech.glide.Glide
10 import
11 com.bumptech.glide.load.resource.bitmap.RoundedCorners
12 import
13 com.example.modul5.data.local.ConstellationEntity
14 import com.example.modul5.databinding.StarItemBinding
15
16 class ListConstellationsAdapter(
17     private val onWebClick: (ConstellationEntity) ->
18     Unit,
19     private val onDetailClick: (ConstellationEntity) ->
20     Unit
21 ) : ListAdapter<ConstellationEntity,
22     ListConstellationsAdapter.ViewHolder>(DIFF_CALLBACK) {
23
24     inner class ViewHolder(val binding:
25     StarItemBinding) :
26     RecyclerView.ViewHolder(binding.root)
27
28     override fun onCreateViewHolder(parent: ViewGroup,
29     viewType: Int): ViewHolder {
30         val binding =
31         StarItemBinding.inflate(LayoutInflater.from(parent.cont
32         ext), parent, false)
33         return ViewHolder(binding)
34     }
35
36     override fun onBindViewHolder(holder: ViewHolder,
37     position: Int) {
38         val item = getItem(position)
39         with(holder.binding) {
40             tvName.text = item.name
41             tvDescription.text = item.description
42             tvYear.text = item.year
43             val radius = TypedValue.applyDimension(
44                 TypedValue.COMPLEX_UNIT_DIP, 16f,
45                 holder.itemView.context.resources.displayMetrics
46                 ).toInt()
47
48             Glide.with(holder.itemView.context)
49                 .load(item.imageUrl)
50                 .transform(RoundedCorners(radius))
51                 .into(holder.binding.img)
52
53             btnWeb.setOnClickListener {
54                 onWebClick(item) }
55             btnDetail.setOnClickListener {
56                 onDetailClick(item) }

```

	<pre> } } companion object { val DIFF_CALLBACK = object : DiffUtil.ItemCallback<ConstellationEntity>() { override fun areItemsTheSame(oldItem: ConstellationEntity, newItem: ConstellationEntity) = oldItem.name == newItem.name override fun areContentsTheSame(oldItem: ConstellationEntity, newItem: ConstellationEntity) = oldItem == newItem } } } </pre>
--	--

10. ui/detail/DetailFragment.kt

Tabel 10. Source Code Jawaban Soal 1 (DetailFragment.kt)

1	package com.example.modul5.ui.detail
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs
9	import com.bumptech.glide.Glide
10	import
11	com.example.modul5.databinding.FragmentDetailBinding
12	
13	class DetailFragment : Fragment() {
14	
15	private var _binding: FragmentDetailBinding? = null
16	private val binding get() = _binding!!
17	private val args: DetailFragmentArgs by navArgs()
18	
19	override fun onCreateView(inflater: LayoutInflater,
20	container: ViewGroup?, savedInstanceState: Bundle?):
21	View {
22	_binding =
23	FragmentDetailBinding.inflate(inflater, container,
24	false)
25	return binding.root
26	}
27	
28	override fun onViewCreated(view: View,
29	savedInstanceState: Bundle?) {
30	val constellation = args.constellation
31	binding.detailName.text = constellation.name

32	binding.detailYear.text = constellation.year
33	binding.detailDescription.text =
34	constellation.description
35	Glide.with(this).load(constellation.imageUrl).into(binding.detailImage)
	}
	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

11. ui/home/HomeFragment.kt

Tabel 11. Source Code Jawaban Soal 1 (HomeFragment.kt)

1	package com.example.modul5.ui.home
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.lifecycleScope
12	import androidx.navigation.fragment.findNavController
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import
15	com.example.modul5.databinding.FragmentHomeBinding
16	import
17	com.example.modul5.ui.adapter.ListConstellationsAdapter
18	import com.example.modul5.viewmodel.MainViewModel
19	import
20	com.example.modul5.viewmodel.MainViewModelFactory
21	import kotlinx.coroutines.flow.collectLatest
22	import kotlinx.coroutines.launch
23	
24	class HomeFragment : Fragment() {
25	
26	private var _binding: FragmentHomeBinding? = null
27	private val binding get() = _binding!!
28	private val viewModel: MainViewModel by viewModels
29	{
30	MainViewModelFactory(requireContext())
31	}
32	
33	override fun onCreateView(inflater: LayoutInflater,
34	container: ViewGroup?, savedInstanceState: Bundle?):

```

35 View {
36     _binding =
37     FragmentHomeBinding.inflate(inflater, container, false)
38     return binding.root
39 }
40
41 override fun onViewCreated(view: View,
42 savedInstanceState: Bundle?) {
43     val adapter = ListConstellationsAdapter(
44         onClick = {
45             val intent = Intent(Intent.ACTION_VIEW,
46 Uri.parse(it.webUrl))
47             startActivity(intent)
48         },
49         onDetailClick = {
50             val action =
51 HomeFragmentDirections.actionHomeFragmentToDetailFragme
52 nt(it)
53             findNavController().navigate(action)
54         }
55     )
56
57     binding.rvStars.layoutManager =
58 LinearLayoutManager(requireContext())
59     binding.rvStars.adapter = adapter
60
61     lifecycleScope.launch {
62         viewModel.state.collectLatest { data ->
63             adapter.submitList(data)
64         }
65     }
66
67     override fun onDestroyView() {
68         super.onDestroyView()
69         _binding = null
70     }
71 }

```

12. ui/MainActivity.kt

Tabel 12. Source Code Jawaban Soal 1 (MainActivity.kt)

```

1 package com.example.modul5.ui
2
3 import android.content.res.Configuration
4 import android.os.Bundle
5 import android.text.SpannableString
6 import android.text.style.ForegroundColorSpan
7 import android.view.Menu
8 import android.view.MenuItem
9 import androidx.appcompat.app.AppCompatActivity

```

```

10 import androidx.appcompat.app.AppCompatActivity
11 import androidx.core.content.ContextCompat
12 import androidx.navigation.fragment.NavHostFragment
13 import
14 androidx.navigation.ui.setupActionBarWithNavController
15 import com.example.modul5.R
16 import
17 com.example.modul5.databinding.ActivityMainBinding
18
19 class MainActivity : AppCompatActivity() {
20
21     private lateinit var binding: ActivityMainBinding
22
23     override fun onCreate(savedInstanceState: Bundle?)
24     {
25         super.onCreate(savedInstanceState)
26
27         binding =
28 ActivityMainBinding.inflate(layoutInflater)
29         setContentView(binding.root)
30         setSupportActionBar(binding.toolbar)
31
32         val navHostFragment = supportFragmentManager
33             .findFragmentById(R.id.nav_host_fragment)
34         as NavHostFragment
35
36         setupActionBarWithNavController(navHostFragment.navCont
37 roller)
38     }
39
40     override fun onCreateOptionsMenu(menu: Menu):
41 Boolean {
42         menuInflater.inflate(R.menu.menu_theme, menu)
43
44         val menuItem =
45 menu.findItem(R.id.action_toggle_theme)
46         val span = SpannableString(menuItem.title)
47
48         span.setSpan(ForegroundColorSpan(ContextCompat.getColor
49 (this, R.color.purple_500)), 0, span.length, 0)
50         menuItem.title = span
51
52         return true
53     }
54
55
56     override fun onOptionsItemSelected(item: MenuItem):
57 Boolean {
58         return when (item.itemId) {
59             R.id.action_toggle_theme -> {
60                 val nightMode =
61 resources.configuration.uiMode and

```

62	Configuration.UI_MODE_NIGHT_MASK
63	val newMode = if (nightMode ==
64	Configuration.UI_MODE_NIGHT_YES)
65	AppCompatActivity.MODE_NIGHT_NO
	else
	AppCompatActivity.MODE_NIGHT_YES
	AppCompatActivity.setDefaultNightMode(newMode)
	true
	}
	else -> super.onOptionsItemSelected(item)
	}
	}
	override fun onSupportNavigateUp(): Boolean {
	val navHostFragment = supportFragmentManager
	.findFragmentById(R.id.nav_host_fragment)
	as NavHostFragment
	return
	navHostFragment.navController.navigateUp()
	super.onSupportNavigateUp()
	}
	}

13. utils/Resource.kt

Tabel 13. Source Code Jawaban Soal 1 (Resource.kt)

1	package com.example.modul5.utils
2	
3	sealed class Resource<out T> {
4	data class Success<out T>(val data: T):
5	Resource<T>()
6	data class Error(val message: String):
7	Resource<Nothing>()
	object Loading : Resource<Nothing>()
	}

14. viewmodel/MainViewModel.kt

Tabel 14. Source Code Jawaban Soal 1 (MainViewModel.kt)

1	package com.example.modul5.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
6	com.example.modul5.data.local.ConstellationEntity
7	import
8	com.example.modul5.data.repository.ConstellationRepository
9	
10	import kotlinx.coroutines.flow.*

11	import kotlinx.coroutines.launch
12	
13	class MainViewModel(
14	private val repository: ConstellationRepository
15	) : ViewModel() {
16	
17	private val _state =
18	MutableStateFlow<List<ConstellationEntity>>(emptyList())
19	)
20	val state: StateFlow<List<ConstellationEntity>> =
21	_state.asStateFlow()
22	
23	init {
24	fetchConstellations()
25	}
26	
27	private fun fetchConstellations() {
28	viewModelScope.launch {
	repository.getConstellations()
	.catch { it.printStackTrace() }
	.collect { _state.value = it }
	}
	}
	}

15. data/viewmodel/MainViewModelFactory.kt

Tabel 15. Source Code Jawaban Soal 1 (MainViewModelFactory.kt)

1	package com.example.modul5.viewmodel
2	
3	import android.content.Context
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	import
7	com.example.modul5.data.local.ConstellationDatabase
8	import
9	com.example.modul5.data.repository.ConstellationRepository
10	
11	
12	class MainViewModelFactory(private val context:
13	Context) : ViewModelProvider.Factory {
14	override fun <T : ViewModel> create(modelClass:
15	Class<T>): T {
16	if
17	(modelClass.isAssignableFrom(MainViewModel::class.java)
18	) {
	val dao =
	ConstellationDatabase.getInstance(context).constellationDao()
	val repo = ConstellationRepository(dao)
	return MainViewModel(repo) as T

	<pre> } throw IllegalArgumentException("Unknown ViewModel class") } } </pre>
--	--

16. ui/AndroidAPIApp.kt

Tabel 16. Source Code Jawaban Soal 1 (AndroidAPIApp.kt)

1	package com.example.modul5
2	
3	import android.app.Application
4	
5	class AndroidAPIApp : Application()

17. res/layout/activity_main.xml

Tabel 17. Source Code Jawaban Soal 1 (activity_main.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.coordinatorlayout.widget.CoordinatorLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent">
9	
10	<com.google.android.material.appbar.AppBarLayout
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	
14	android:theme="@style/ThemeOverlay.MaterialComponents.D
15	ark.ActionBar">
16	
17	<androidx.appcompat.widget.Toolbar
18	android:id="@+id/toolbar"
19	android:layout_width="match_parent"
20	android:layout_height="?attr/actionBarSize"
21	android:background="?attr/colorPrimary"
22	android:elevation="4dp"
23	app:titleTextColor="@android:color/white"
24	
25	</com.google.android.material.appbar.AppBarLayout>
26	
27	<androidx.fragment.app.FragmentContainerView
28	android:id="@+id/nav_host_fragment"
29	
30	android:name="androidx.navigation.fragment.NavHostFragm

	<pre> ent" android:layout_width="match_parent" android:layout_height="match_parent" app:navGraph="@navigation/nav_graph" app:defaultNavHost="true" android:layout_marginTop="?attr/actionBarSize"/> </androidx.coordinatorlayout.widget.CoordinatorLayout> </pre>
--	---

18. res/layout/fragment_detail.xml

Tabel 18. Source Code Jawaban Soal 1 (fragment_detail.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	android:id="@+id/detailFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	android:padding="16dp">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="vertical"
15	android:gravity="center_horizontal">
16	
17	<ImageView
18	android:id="@+id/detailImage"
19	android:layout_width="300dp"
20	android:layout_height="300dp"
21	android:scaleType="centerCrop"
22	android:layout_marginBottom="16dp" />
23	
24	<TextView
25	android:id="@+id/detailName"
26	android:layout_width="wrap_content"
27	android:layout_height="wrap_content"
28	android:text="Name"
29	android:textSize="22sp"
30	android:textStyle="bold"
31	android:layout_marginBottom="8dp" />
32	
33	<TextView
34	android:id="@+id/detailYear"
35	android:layout_width="wrap_content"
36	android:layout_height="wrap_content"
37	android:text="Year"
38	android:textSize="16sp"
39	android:layout_marginBottom="16dp" />

40	
41	<TextView
42	android:id="@+id/detailDescription"
43	android:layout_width="match_parent"
44	android:layout_height="wrap_content"
45	android:text="Description"
46	android:textSize="16sp" />
	</LinearLayout>
	</ScrollView>

19. res/layout/fragment_home.xml

Tabel 19. Source Code Jawaban Soal 1 (fragment_home.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	android:id="@+id/homeFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	android:padding="8dp">
10	
11	<androidx.recyclerview.widget.RecyclerView
12	android:id="@+id/rvStars"
13	android:layout_width="match_parent"
14	android:layout_height="match_parent"
15	android:clipToPadding="false" />
	</FrameLayout>

20. res/layout/star_item.xml

Tabel 20. Source Code Jawaban Soal 1 (star_item.xml)

1	package com.example.modul5.data.local
2	<?xml version="1.0" encoding="utf-8"?>
3	<LinearLayout
4	
5	xmlns:android="http://schemas.android.com/apk/res/andr
6	oid"
7	xmlns:app="http://schemas.android.com/apk/res-
8	auto"
9	android:layout_width="match_parent"
10	android:layout_height="wrap_content"
11	android:orientation="vertical"
12	android:padding="8dp">
13	
14	<androidx.cardview.widget.CardView
15	android:layout_width="match_parent"
16	android:layout_height="wrap_content"

```

17         android:layout_margin="8dp"
18         app:cardCornerRadius="12dp"
19         app:cardElevation="4dp">
20
21         <LinearLayout
22             android:layout_width="match_parent"
23             android:layout_height="wrap_content"
24             android:orientation="horizontal"
25             android:padding="8dp">
26
27             <ImageView
28                 android:id="@+id/img"
29                 android:layout_width="100dp"
30                 android:layout_height="150dp"
31                 android:scaleType="centerCrop"
32                 android:layout_marginEnd="8dp"             />
33
34             <LinearLayout
35                 android:layout_width="0dp"
36                 android:layout_height="match_parent"
37                 android:layout_weight="1"
38                 android:orientation="vertical">
39
40                 <LinearLayout
41
42                     android:layout_width="match_parent"
43
44                     android:layout_height="wrap_content"
45                         android:orientation="horizontal"
46                         android:gravity="center_vertical">
47
48                         <TextView
49                             android:id="@+id/tvName"
50                             android:layout_width="0dp"
51
52                             android:layout_height="wrap_content"
53                             android:layout_weight="1"
54                             android:text="Constellations
55                             Name"
56
57                             android:textStyle="bold"
58                             android:textSize="16sp"             />
59
60                             <TextView
61                                 android:id="@+id/tvYear"
62
63                                 android:layout_width="wrap_content"
64
65                                 android:layout_height="wrap_content"
66                                 android:text="Year"
67                                 android:textSize="14sp"             />
68                 </LinearLayout>

```

69	<TextView
70	android:id="@+id/tvDescription"
71	
72	android:layout_width="match_parent"
73	
74	android:layout_height="wrap_content"
75	android:text="Description"
76	android:maxLines="3"
77	android:ellipsize="end"
78	android:textSize="14sp"
79	android:layout_marginTop="4dp" />
80	
81	<LinearLayout
82	
83	android:layout_width="match_parent"
84	
85	android:layout_height="wrap_content"
86	android:orientation="horizontal"
87	android:layout_marginTop="8dp">
88	
89	
90	<com.google.android.material.button.MaterialButton
91	android:id="@+id/btnWeb"
92	
93	style="@style/Widget.MaterialComponents.Button"
94	android:layout_width="0dp"
95	
96	android:layout_height="wrap_content"
97	android:layout_weight="1"
98	android:text="Website"
99	android:textAllCaps="false"
101	app:cornerRadius="50dp"
102	app:iconPadding="0dp" />
	<com.google.android.material.button.MaterialButton
	android:id="@+id/btnDetail"
	style="@style/Widget.MaterialComponents.Button"
	android:layout_width="0dp"
	android:layout_height="wrap_content"
	android:layout_weight="1"
	android:text="Detail"
	android:textAllCaps="false"
	app:cornerRadius="50dp"
	android:layout_marginStart="8dp"
	app:iconPadding="0dp" />
	</LinearLayout>
	</LinearLayout>

	<pre> </LinearLayout> </androidx.cardview.widget.CardView> </LinearLayout> </pre>
--	---

21. res/menu/menu_theme.xml

Tabel 21. Source Code Jawaban Soal 1 (menu_theme.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu
3	xmlns:android="http://schemas.android.com/apk/res/android
4	"
5	xmlns:app="http://schemas.android.com/apk/res-auto">
6	<item
7	android:id="@+id/action_toggle_theme"
8	android:title="Light/Dark Mode"
	app:showAsAction="never" />
	</menu>

22. res/navigation/navigation_graph.xml

Tabel 22. Source Code Jawaban Soal 1 (navigation_graph.xml)

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	
4	xmlns:android="http://schemas.android.com/apk/res/andro
5	id"
6	xmlns:app="http://schemas.android.com/apk/res-auto"
7	android:id="@+id/nav_graph"
8	app:startDestination="@id/homeFragment">
9	
10	<fragment
11	android:id="@+id/homeFragment"
12	
13	android:name="com.example.modul5.ui.home.HomeFragment"
14	android:label="Home" >
15	<action
16	
17	android:id="@+id/action_homeFragment_to_detailFragment"
18	app:destination="@id/detailFragment" />
19	</fragment>
20	
21	<fragment
22	android:id="@+id/detailFragment"
23	
24	android:name="com.example.modul5.ui.detail.DetailFragme
25	nt"
	android:label="Detail">
	<argument
	android:name="constellation"

	<pre> app:argType="com.example.modul5.data.local.Constellatio nEntity" /> </fragment> </navigation> </pre>
--	---

23. res/values/themes/themes.xml

Tabel 23. Source Code Jawaban Soal 1 (themes.xml)

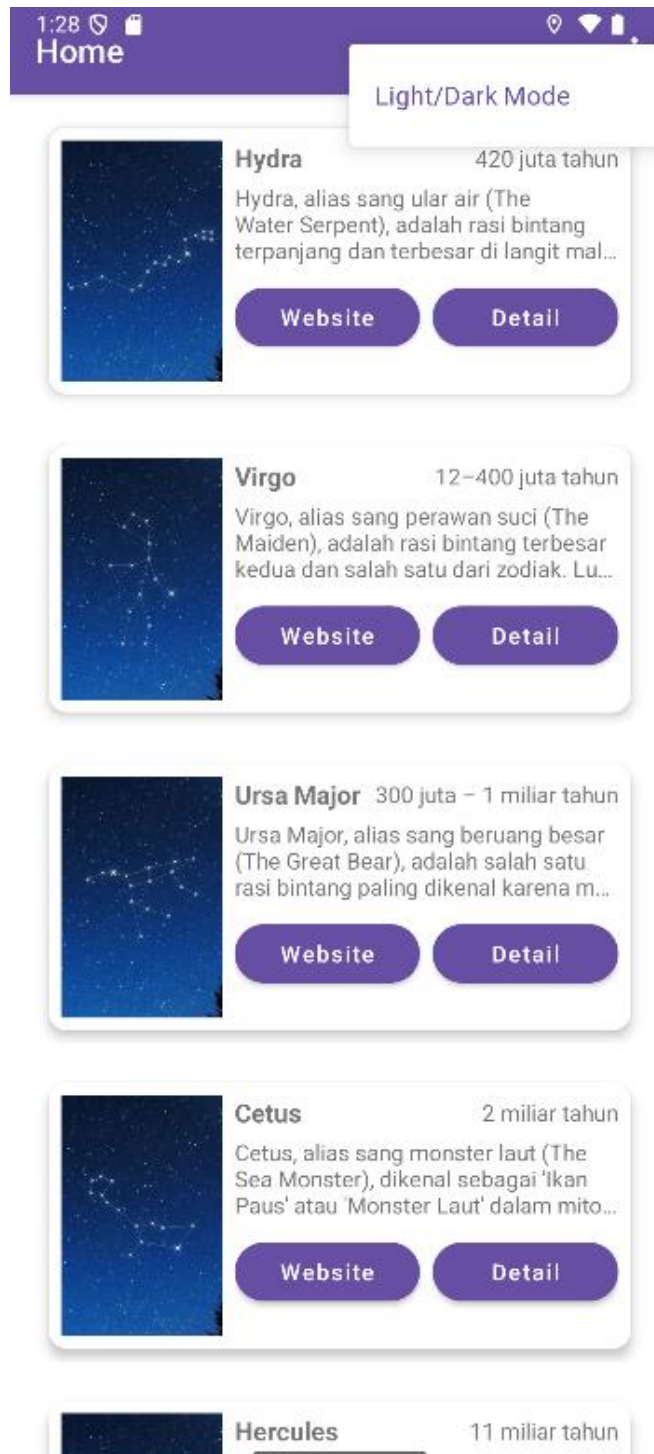
1	<resources
2	xmlns:tools="http://schemas.android.com/tools">
3	<style name="Theme.MODUL5"
4	parent="Theme.MaterialComponents.DayNight.NoActionBar">
5	<item
6	name="colorPrimary">@color/purple_500</item>
7	<item
8	name="colorPrimaryVariant">@color/purple_700</item>
9	<item
10	name="colorOnPrimary">@android:color/white</item>
	<item
	name="colorSecondary">@color/teal_200</item>
	<item
	name="colorSecondaryVariant">@color/teal_700</item>
	<item
	name="colorOnSecondary">@android:color/white</item>
	</style>
	</resources>

24. res/values/themes/themes.xml (night)

Tabel 24. Source Code Jawaban Soal 1 (themes.xml (night))

1	<resources
2	xmlns:tools="http://schemas.android.com/tools">
3	<style name="Theme.MODUL5"
4	parent="Theme.MaterialComponents.DayNight.NoActionBar">
5	<item
6	name="colorPrimary">@color/purple_200</item>
7	<item
8	name="colorPrimaryVariant">@color/purple_700</item>
9	<item
10	name="colorOnPrimary">@android:color/white</item>
	<item
	name="colorSecondary">@color/teal_200</item>
	<item
	name="colorSecondaryVariant">@color/teal_700</item>
	<item
	name="colorOnSecondary">@android:color/white</item>
	</style>
	</resources>

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 (Home Light Mode)

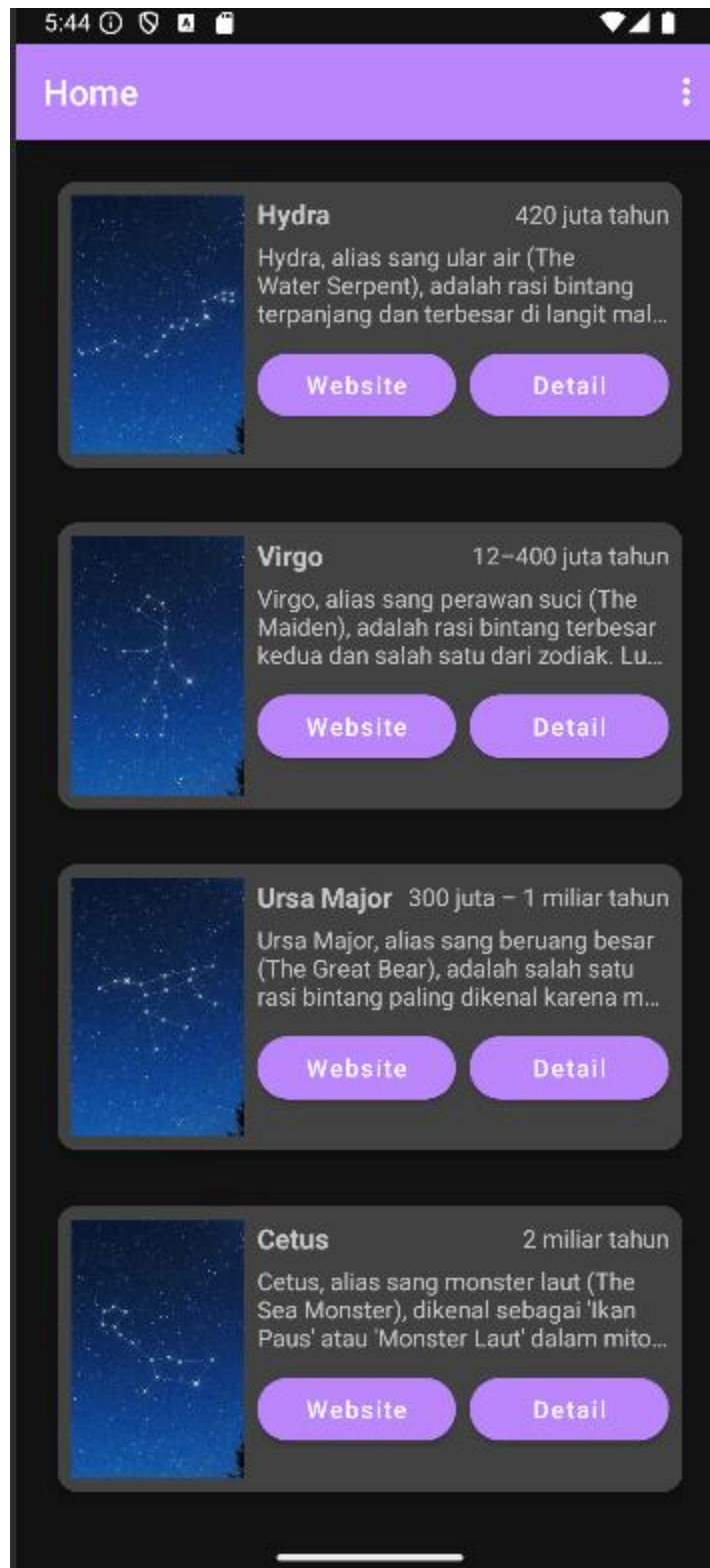


Hydra

420 juta tahun

Hydra, alias sang ular air (The Water Serpent), adalah rasi bintang terpanjang dan terbesar di langit malam. Luasnya sebesar 1.303 derajat persegi (~3,16% dari seluruh langit malam). Bentuknya menyerupai ular air dan membentang dari selatan Cancer hingga Libra. Meski ukurannya besar, hanya sedikit bintang terang yang terlihat. Hydra dikenal dalam mitologi Yunani sebagai ular berkepala banyak yang dikalahkan oleh Herkules.

Gambar 2. Screenshot Hasil Jawaban Soal 1 (Detail Light Mode)



Gambar 3. Screenshot Hasil Jawaban Soal 1 (Home Dark Mode)



Gambar 4. Screenshot Hasil Jawaban Soal 1 (Detail Dark Mode)



Gambar 5. Screenshot Hasil Jawaban Soal 1 (Website)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. data/local/ConstellationDao.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`
- Pada line 3, mengimpor anotasi dan class dari Room seperti `Dao`, `Insert`, `OnConflictStrategy`, dan `Query`.
- Pada line 4, mengimpor `Flow` dari `kotlinx.coroutines.flow` untuk mendukung data yang mengalir secara asinkron.
- Pada line 6, mendeklarasikan interface `ConstellationDao` dan menandainya sebagai DAO dengan anotasi `@Dao`.
- Pada line 9, mendeklarasikan fungsi `getAll()` dengan anotasi `@Query`, untuk mengambil seluruh data dari tabel `constellations`, dan mengembalikannya dalam bentuk `Flow<List<ConstellationEntity>>`.
- Pada line 11, mendeklarasikan fungsi `insertAll()` dengan anotasi `@Insert`, untuk menyimpan data list `ConstellationEntity`, dan menggunakan strategi `REPLACE` jika ada konflik primary key.

2. data/local/ConstellationDatabase.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`.
- Pada line 3, mengimpor `Context` dari Android untuk digunakan saat inisialisasi database.
- Pada line 4, mengimpor anotasi `Database` dari Room untuk mendefinisikan konfigurasi database.
- Pada line 5, mengimpor class `Room` untuk membangun instance database.
- Pada line 6, mengimpor class `RoomDatabase` sebagai superclass untuk database Room.
- Pada line 8, mendeklarasikan anotasi `@Database` dengan entitas `ConstellationEntity`, versi 1, dan tanpa mengekspos skema.

- Pada line 9, mendeklarasikan class abstract `ConstellationDatabase` yang mewarisi `RoomDatabase`.
- Pada line 10, mendeklarasikan fungsi abstrak `constellationDao()` untuk mengakses DAO dari database.
- Pada line 12, mendeklarasikan blok companion object untuk menyimpan instance singleton database.
- Pada line 13, mendeklarasikan variabel `INSTANCE` bertipe nullable `ConstellationDatabase`, dan ditandai dengan `@Volatile` agar aman untuk akses dari banyak thread.
- Pada line 15, mendeklarasikan fungsi `getInstance()` untuk mengembalikan instance database, menerima parameter `context`.
- Pada line 16, menggunakan operator Elvis (`?:`) untuk mengecek apakah `INSTANCE` sudah tersedia, jika belum maka lanjut membuat instance baru.
- Pada line 17-20, membuat instance database menggunakan `Room.databaseBuilder`, dengan konteks aplikasi, referensi class database, dan nama database `"constellation_db"`.
- Pada line 20, menyimpan instance yang telah dibuat ke dalam variabel `INSTANCE` dan mengembalikannya.

3. `data/local/Constellation Entity.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.local`.
- Pada line 3, mengimpor interface `Parcelable` dari Android untuk mendukung passing objek antar komponen.
- Pada line 4, mengimpor anotasi `Entity` dari Room untuk mendefinisikan class sebagai entitas tabel database.
- Pada line 5, mengimpor anotasi `PrimaryKey` dari Room untuk menandai atribut primary key.
- Pada line 6, mengimpor anotasi `Parcelize` dari Kotlin untuk mengotomatisasi implementasi `Parcelable`.

- Pada line 8, menandai class `ConstellationEntity` dengan anotasi `@Parcelize` agar dapat di-parcel secara otomatis.
- Pada line 9, menandai class `ConstellationEntity` sebagai entitas Room dengan nama tabel `constellations`.
- Pada line 10, mendeklarasikan properti `name` sebagai primary key dan bertipe `String`.
- Pada line 11-14, mendeklarasikan properti tambahan: `description`, `year`, `imageUrl`, dan `webUrl`, semuanya bertipe `String`.
- Pada line 15, menandai bahwa class `ConstellationEntity` mengimplementasikan `Parcelable`.

4. data/model/ApiResponse.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.model`.
- Pada line 3, mengimpor anotasi `Serializable` dari Kotlinx Serialization untuk mendukung serialisasi data.
- Pada line 5, menandai data class `ApiResponse` sebagai serializable menggunakan anotasi `@Serializable`.
- Pada line 6-8, mendeklarasikan properti `status`, `message`, dan data bertipe generic `T`, yang merepresentasikan format response dari API.

5. data/model/Constellation.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.model`.
- Pada line 3, mengimpor anotasi `SerializedName` dari Kotlinx Serialization untuk mapping nama field JSON.
- Pada line 4, mengimpor anotasi `Serializable` dari Kotlinx Serialization.
- Pada line 6, menandai data class `Constellation` sebagai serializable menggunakan anotasi `@Serializable`.

- Pada line 7-9, mendeklarasikan properti `name`, `description`, dan `year` bertipe `String`.
- Pada line 10, menggunakan anotasi `@SerializedName("image_url")` untuk memetakan JSON field `image_url` ke properti `imageUrl`.
- Pada line 11, menggunakan anotasi `@SerializedName("web_url")` untuk memetakan JSON field `web_url` ke properti `webUrl`.

6. `data/remote/ApiService.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.remote`.
- Pada line 3, mengimpor model `ApiResponse` dari package `data.model`.
- Pada line 4, mengimpor model `Constellation` dari package `data.model`.
- Pada line 5, mengimpor anotasi `GET` dari Retrofit untuk request HTTP GET.
- Pada line 7, mendeklarasikan interface `ApiService` untuk definisi endpoint API.
- Pada line 8, menggunakan anotasi `@GET("constellation")` untuk mendefinisikan endpoint GET ke path `constellation`.
- Pada line 9, mendeklarasikan fungsi `getConstellations()` sebagai fungsi `suspending` yang mengembalikan `ApiResponse<List<Constellation>>`.

7. `data/remote/RetrofitInstance.kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.remote`.
- Pada line 3, mengimpor converter factory dari `kotlinx.serialization` untuk digunakan oleh Retrofit.
- Pada line 4, mengimpor `Json` dari Kotlinx Serialization sebagai konfigurasinya.
- Pada line 5, mengimpor `toMediaType` dari `okhttp3` untuk menentukan jenis media request.
- Pada line 6, mengimpor `OkHttpClient` untuk membuat client HTTP.

- Pada line 7, mengimpor `HttpLoggingInterceptor` untuk log detail request dan response HTTP.
- Pada line 8, mengimpor `Retrofit` sebagai builder HTTP client utama.
- Pada line 10, mendeklarasikan objek singleton `RetrofitInstance`.
- Pada line 12, mendeklarasikan constant `BASE_URL` sebagai base URL API.
- Pada line 14-16, menginisialisasi objek `Json` dengan pengaturan `ignoreUnknownKeys = true` untuk mengabaikan field JSON yang tidak dikenali.
- Pada line 18-21, menginisialisasi objek `OkHttpClient` dan menambahkan `HttpLoggingInterceptor` dengan level log BODY.
- Pada line 23, mendeklarasikan properti `api` bertipe `ApiService` yang dibuat secara lazy.
- Pada line 24-28, membangun objek `Retrofit` dengan `baseUrl`, converter untuk JSON, client, dan kemudian membuat instance `ApiService`.

8. data/repository/ConstellationRepository.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.data.repository`.
- Pada line 3, mengimpor `ConstellationDao` dari local data source.
- Pada line 4, mengimpor model `ConstellationEntity` sebagai data yang akan disimpan ke database lokal.
- Pada line 5, mengimpor `RetrofitInstance` untuk melakukan request ke API.
- Pada line 6, mengimpor `Flow` dari Kotlin untuk mengalirkan data secara reaktif.
- Pada line 7, mengimpor fungsi `flow` dari Kotlin untuk membuat objek `Flow` baru.
- Pada line 8, mengimpor `emitAll` untuk meneruskan aliran data dari DAO ke UI.
- Pada line 10, mendeklarasikan class `ConstellationRepository` dengan parameter `dao` sebagai instance dari `ConstellationDao`.

- Pada line 12, mendefinisikan fungsi `getConstellations()` yang mengembalikan `Flow<List<ConstellationEntity>>`.
- Pada line 13, menggunakan `flow {}` untuk membuat aliran data yang bisa dikoleksi.
- Pada line 14, membuat blok try-catch untuk menghindari error saat fetch data.
- Pada line 15, memanggil fungsi `getConstellations()` dari API dan menyimpannya ke variabel `response`.
- Pada line 16, melakukan pengecekan apakah `status` dari `response` bernilai "success".
- Pada line 17-23, melakukan mapping data dari `response` API ke `ConstellationEntity`, lalu menyimpannya ke database lokal dengan `dao.insertAll(mapped)`.
- Pada line 24-25, menangani exception dengan mencetak stack trace.
- Pada line 27, meneruskan seluruh data dari DAO dengan `emitAll(dao.getAll())`.

9. `ui/adapter/ListConstellation Adapter kt`

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.adapter`.
- Pada line 3-5, mengimpor class yang dibutuhkan untuk manipulasi tampilan dan pengaturan ukuran.
- Pada line 6-8, mengimpor class dari `RecyclerView` untuk pembuatan adapter dan perbandingan data.
- Pada line 9-10, mengimpor `Glide` dan `RoundedCorners` untuk pemuatan dan pengubahan tampilan gambar.
- Pada line 11, mengimpor `ConstellationEntity` sebagai model data untuk list item.
- Pada line 12, mengimpor binding layout `StarItemBinding`.

- Pada line 14-18, mendeklarasikan class `ListConstellationsAdapter` yang mewarisi `ListAdapter`, serta menerima dua lambda `onWebClick` dan `onDetailClick`.
- Pada line 20, membuat inner class `ViewHolder` untuk menyimpan objek binding per item.
- Pada line 22-24, mengimplementasikan `onCreateViewHolder` untuk meng-inflate layout dan membuat instance `ViewHolder`.
- Pada line 26-41, mengimplementasikan `onBindViewHolder` untuk menampilkan data dan menangani klik tombol.
- Pada line 27-30, mengisi `TextView` dengan nama, deskripsi, dan tahun dari item.
- Pada line 31-33, menghitung ukuran radius sudut gambar dalam satuan dp ke px.
- Pada line 35-37, memuat gambar menggunakan `Glide` dan menerapkan efek sudut membulat.
- Pada line 40-41, menangani event klik pada tombol Web dan tombol Detail.
- Pada line 43-50, membuat objek `DIFF_CALLBACK` untuk membandingkan item list berdasarkan nama dan isi secara menyeluruh.

10. ui/detail/DetailFragment.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.detail`.
- Pada line 3-5, mengimpor class `Android` untuk pengelolaan fragment dan view.
- Pada line 6, mengimpor `navArgs` untuk mengambil argumen navigasi antar fragment.
- Pada line 7, mengimpor `Glide` untuk pemrosesan gambar.
- Pada line 8, mengimpor binding `FragmentDetailBinding`.
- Pada line 10, mendeklarasikan class `DetailFragment` yang mewarisi `Fragment`.
- Pada line 12, mendeklarasikan variabel `_binding` yang bersifat nullable.

- Pada line 13, mendeklarasikan properti `binding` yang menggunakan `!!` untuk memastikan `_binding` tidak null.
- Pada line 14, mendeklarasikan properti `args` untuk menerima data dari navigasi.
- Pada line 16-18, meng-inflate layout fragment menggunakan `FragmentDetailBinding`.
- Pada line 20-25, mengisi komponen UI dengan data dari argumen `constellation` dan memuat gambar menggunakan `Glide`.
- Pada line 27-29, mengatur `_binding` menjadi null saat view dihancurkan untuk mencegah memory leak.

11. ui/home/HomeFragment.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui.home`.
- Pada line 3-5, mengimpor class Android untuk keperluan navigasi dan intent.
- Pada line 6-9, mengimpor class fragment, view model, dan navigasi dari `AndroidX`.
- Pada line 10, mengimpor `LinearLayoutManager` untuk mengatur layout `RecyclerView`.
- Pada line 11, mengimpor binding layout `FragmentHomeBinding`.
- Pada line 12, mengimpor `ListConstellationsAdapter` untuk menampilkan list.
- Pada line 13-14, mengimpor `MainViewModel` dan `MainViewModelFactory` sebagai penyedia data.
- Pada line 15-16, mengimpor `Flow` dan `coroutine` untuk mengoleksi data dari `ViewModel`.
- Pada line 18, mendeklarasikan class `HomeFragment` yang mewarisi `Fragment`.
- Pada line 20, mendeklarasikan variabel `_binding` untuk binding layout fragment.

- Pada line 21, membuat properti binding dari `_binding` dengan asumsi tidak null.
- Pada line 22-25, mendeklarasikan dan menginisialisasi `viewModel` menggunakan `viewModels` dengan `MainViewModelFactory`.
- Pada line 27-29, meng-inflate layout `FragmentHomeBinding` saat fragment dibuat.
- Pada line 31-48, mengatur tampilan list dengan adapter dan menghubungkannya dengan data dari `ViewModel`.
- Pada line 32-39, menginisialisasi adapter dan mengatur navigasi tombol Web dan Detail.
- Pada line 41-42, mengatur layout manager dan adapter untuk `RecyclerView`.
- Pada line 44-47, meluncurkan coroutine untuk mengoleksi data terbaru dari `ViewModel` dan mengirimkannya ke adapter.
- Pada line 50-52, membersihkan binding saat view dihancurkan.

12. ui/MainActivity.kt

- Pada line 1, mendeklarasikan nama package untuk file Kotlin, yaitu `com.example.modul5.ui`.
- Pada line 3-5, mengimpor class `Android` untuk konfigurasi tema dan tampilan teks menu.
- Pada line 6-8, mengimpor class `AppCompatActivity`, tema, dan akses warna.
- Pada line 9, mengimpor `NavHostFragment` dari library navigasi.
- Pada line 10, mengimpor `setupActionBarWithNavController` untuk integrasi action bar.
- Pada line 11, mengimpor `R` untuk mengakses resource.
- Pada line 12, mengimpor binding layout `ActivityMainBinding`.
- Pada line 14, mendeklarasikan class `MainActivity` yang mewarisi `AppCompatActivity`.
- Pada line 16, mendeklarasikan properti binding untuk akses komponen UI..

- Pada line 18-23, menginisialisasi `binding`, menyetel `layout` sebagai konten, dan mengatur `toolbar` sebagai `action bar`.
- Pada line 25-27, mendapatkan `NavHostFragment` dan menyetel `controller` untuk navigasi.
- Pada line 29-37, meng-override `onCreateOptionsMenu` untuk menampilkan item menu tema dan memberi warna khusus pada teksnya.
- Pada line 40-48, meng-override `onOptionsItemSelected` untuk menangani perubahan tema antara `light` dan `dark mode` saat menu ditekan.
- Pada line 50-53, meng-override `onSupportNavigateUp` untuk menangani tombol navigasi kembali di `action bar`.

13. `utils/Resource.kt`

- Pada line 1, mendeklarasikan `package com.example.modul5.utils`.
- Pada line 3, mendeklarasikan `class Resource` sebagai `sealed class` yang mewakili tiga status: **Success**, **Error**, dan **Loading**.
- Pada line 4, data `class Success` membawa data bertipe `T`, menunjukkan permintaan berhasil.
- Pada line 5, data `class Error` membawa pesan error, tanpa data (`Nothing`).
- Pada line 6, object `Loading` digunakan saat proses sedang berjalan (juga bertipe `Resource<Nothing>`).

14. `viewmodel/MainViewModel.kt`

- Pada line 1, mendeklarasikan `package com.example.modul5.viewmodel`.
- Pada line 3-4, mengimpor `ViewModel` dan `viewModelScope` dari `AndroidX Lifecycle`.
- Pada line 5-6, mengimpor `ConstellationEntity` dan `ConstellationRepository`.
- Pada line 7-8, mengimpor `Flow` dan `coroutine` dari `kotlinx.coroutines`.

- Pada line 10, mendeklarasikan class `MainViewModel` yang menerima repository dan mewarisi `ViewModel`.
- Pada line 12, mendeklarasikan `_state` sebagai `MutableStateFlow` dengan nilai awal list kosong.
- Pada line 13, mengekspos `state` sebagai `StateFlow` agar hanya bisa dibaca oleh luar kelas.
- Pada line 15, memanggil `fetchConstellations()` saat `ViewModel` pertama kali dibuat.
- Pada line 17-22, mendefinisikan fungsi `fetchConstellations()` untuk mengambil data dari repository:
- Pada line 18, menjalankan coroutine dalam scope `ViewModel`.
- Pada line 19, memanggil `getConstellations()` dari repository.
- Pada line 20, mencetak error ke log jika terjadi kesalahan.
- Pada line 21, mengisi `state` dengan hasil yang dikoleksi dari flow.

15. `viewmodel/MainViewModelFactory.kt`

- Pada line 1, mendeklarasikan package `com.example.modul5.viewmodel`.
- Pada line 3-4, mengimpor konteks Android dan class untuk membuat `ViewModel`.
- Pada line 5-6, mengimpor database dan repository.
- Pada line 8, mendeklarasikan class `MainViewModelFactory` yang menerima `Context`.
- Pada line 9, meng-override fungsi `create()` dari `ViewModelProvider.Factory`.
- Pada line 10, memeriksa apakah `modelClass` cocok dengan `MainViewModel`.
- Pada line 11, mengambil DAO dari singleton `ConstellationDatabase`.
- Pada line 12, membuat instance `ConstellationRepository` menggunakan DAO.

- Pada line 13, mengembalikan `MainViewModel` dengan repository sebagai argumen.
- Pada line 15, melempar error jika class `ViewModel` tidak dikenal.

16. `com.example.modul5/AndroidAPIApp.kt`

- Pada line 1, mendeklarasikan package utama aplikasi: `com.example.modul5`.
- Pada line 3, mengimpor `Application` dari Android.
- Pada line 5, mendeklarasikan class `AndroidAPIApp` yang mewarisi `Application`, digunakan untuk inisialisasi global (misalnya Room atau library lain di masa depan).

17. `res/layout/activity_main.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5–6, terdapat `layout_width` dan `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8–14, terdapat elemen `AppBarLayout` yang digunakan untuk membungkus toolbar dan mendukung scroll behavior:
- `layout_width` dan `layout_height` mengikuti layar dan konten.
- `android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"` memberi tampilan tema gelap pada AppBar.
- Pada line 10–13, terdapat `Toolbar` yang digunakan sebagai action bar:
- `layout_height="@attr/actionBarSize"` mengikuti tinggi standar action bar.
- `background="@attr/colorPrimary"` mengikuti warna utama dari tema.
- `elevation="4dp"` memberi efek bayangan di bawah toolbar.

- `app:titleTextColor="@android:color/white"` membuat warna teks judul toolbar menjadi putih.
- Pada line 16–23, terdapat `FragmentContainerView` yang digunakan sebagai tempat menampilkan fragment:
- `android:name="androidx.navigation.fragment.NavHostFragment"` menjadikan komponen ini sebagai host fragment dari `Navigation Component`.
- `app:navGraph="@navigation/nav_graph"` menunjuk ke file navigasi `nav_graph.xml`.
- `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama aplikasi.
- `android:layout_marginTop="?attr/actionBarSize"` memberi margin atas agar tidak tertutup oleh toolbar.
- Pada line 25, penutup `CoordinatorLayout`.

18. `res/layout/fragment_detail.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–25, terdapat elemen root `CoordinatorLayout` dipakai untuk mendukung layout berbasis koordinasi antar elemen, seperti toolbar dan fragment. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 5–6, terdapat `layout_width` dan `layout_height` yang di-set ke `match_parent`, artinya mengisi seluruh layar.
- Pada line 8–14, terdapat elemen `AppBarLayout` yang digunakan untuk membungkus toolbar dan mendukung scroll behavior:
 - `layout_width` dan `layout_height` mengikuti layar dan konten.
 - `android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar"` memberi tampilan tema gelap pada `AppBar`.
- Pada line 10–13, terdapat `Toolbar` yang digunakan sebagai action bar:
 - `layout_height="?attr/actionBarSize"` mengikuti tinggi standar action bar.

- `background="?attr/colorPrimary"` mengikuti warna utama dari tema.
- `elevation="4dp"` memberi efek bayangan di bawah toolbar.
- `app:titleTextColor="@android:color/white"` membuat warna teks judul toolbar menjadi putih.
- Pada line 16–23, terdapat `FragmentContainerView` yang digunakan sebagai tempat menampilkan fragment:
 - `android:name="androidx.navigation.fragment.NavHostFragment"` menjadikan komponen ini sebagai host fragment dari Navigation Component.
 - `app:navGraph="@navigation/nav_graph"` menunjuk ke file navigasi `nav_graph.xml`.
 - `app:defaultNavHost="true"` menjadikan fragment ini sebagai host utama aplikasi.
 - `android:layout_marginTop="?attr/actionBarSize"` memberi margin atas agar tidak tertutup oleh toolbar.
- Pada line 25, penutup `CoordinatorLayout`.

19. `res/layout/fragment_home.xml`

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–10, terdapat elemen root `FrameLayout` sebagai kontainer utama untuk layout fragment home.
- Pada line 3, `android:id="@+id/homeFragment"` digunakan untuk mengidentifikasi fragment di kode.
- Pada line 4–5, `layout_width` dan `layout_height` di-set ke `match_parent` agar mengisi seluruh layar.
- Pada line 6, `android:padding="8dp"` memberikan jarak di seluruh sisi dalam layout.
- Pada line 8, terdapat elemen `RecyclerView` (id: `rvStars`) sebagai komponen utama untuk menampilkan daftar konstelasi secara scrollable:

- `layout_width` dan `layout_height` di-set ke `match_parent` untuk mengisi layout.
- `android:clipToPadding="false"` memungkinkan isi recycler view tetap terlihat meski melebihi batas padding.
- Pada line 10, penutup `FrameLayout`.

20. res/layout/star_item.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–64, terdapat elemen root `LinearLayout` vertikal yang menjadi pembungkus setiap item konstelasi dalam daftar.
- Pada line 3–4, terdapat namespace `xmlns:android` dan `xmlns:app` agar atribut XML dikenali.
- Pada line 5–6, `layout_width="match_parent"` dan `layout_height="wrap_content"` membuat item menyesuaikan tinggi konten dan selebar layar.
- Pada line 7, `android:padding="8dp"` memberikan jarak dalam layout item.
- Pada line 9–59, terdapat elemen `CardView` sebagai kartu tampilan item konstelasi:
 - `cardCornerRadius="12dp"` memberikan sudut membulat.
 - `cardElevation="4dp"` memberikan efek bayangan agar tampil menonjol.
- Pada line 11–57, terdapat `LinearLayout` horizontal yang membungkus gambar dan teks.
- Pada line 13–16, terdapat `ImageView` (id: `img`) untuk menampilkan gambar konstelasi:
 - `layout_width="100dp"` dan `layout_height="150dp"` membuat gambar berukuran proporsional.
 - `scaleType="centerCrop"` untuk menjaga proporsi gambar.
 - `layout_marginEnd="8dp"` memberi jarak dari teks.
- Pada line 18–54, terdapat `LinearLayout` vertikal yang berisi informasi teks dan tombol:
 - `layout_weight="1"` agar mengambil sisa ruang horizontal.

- Pada line 20–25, terdapat `LinearLayout` horizontal untuk nama dan tahun konstelasi:
 - `gravity="center_vertical"` meratakan isi secara vertikal di tengah.
- Pada line 22, `TextView` (id: `tvName`) untuk menampilkan nama konstelasi:
 - `layout_weight="1"` membuatnya mengisi sisa ruang.
 - `textStyle="bold"` dan `textSize="16sp"` membuat teks tebal dan jelas.
- Pada line 24, `TextView` (id: `tvYear`) untuk menampilkan tahun konstelasi dengan `textSize="14sp"`.
- Pada line 27–30, `TextView` (id: `tvDescription`) untuk menampilkan deskripsi pendek:
 - `maxLines="3"` dan `ellipsize="end"` digunakan agar teks tidak terlalu panjang dan terpotong dengan titik-titik.
- Pada line 32–52, terdapat `LinearLayout` horizontal untuk dua tombol aksi:
- Pada line 34–38, `MaterialButton` (id: `btnWeb`) digunakan untuk membuka tautan ke website konstelasi:
 - `layout_weight="1"` agar lebar tombol seimbang dengan tombol lain.
 - `text="Website"` dengan `cornerRadius="50dp"` membuat tampilan membulat.
- Pada line 40–46, `MaterialButton` (id: `btnDetail`) untuk navigasi ke detail konstelasi:
 - `layout_marginStart="8dp"` memberi jarak dari tombol sebelumnya.
 - `text="Detail"` dan `cornerRadius="50dp"` membuat tombol tampil konsisten.
- Pada line 64, penutup `LinearLayout`.

21. res/menu/menu_theme.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.

- Pada line 2–7, terdapat elemen root `<menu>` yang digunakan untuk mendefinisikan item menu di action bar atau toolbar. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 3–6, terdapat elemen `<item>` untuk menampilkan menu switch tema:
 - `android:id="@+id/action_toggle_theme"` memberikan ID unik untuk aksi toggle.
 - `android:title="Light/Dark Mode"` menampilkan teks judul pada menu.
 - `app:showAsAction="never"` artinya item tidak ditampilkan sebagai ikon di toolbar, hanya muncul di overflow menu.
- Pada line 7, penutup elemen `<menu>`.

22. res/navigation/navigation_graph.xml

- Pada line 1, terdapat deklarasi XML dan encoding UTF-8.
- Pada line 2–19, terdapat elemen root `<navigation>` yang digunakan untuk mendefinisikan arah navigasi antar fragment menggunakan Jetpack Navigation Component. Namespace `xmlns:android` dan `xmlns:app` digunakan agar atribut XML dikenali.
- Pada line 3, `android:id="@+id/nav_graph"` memberikan ID untuk graph ini.
- Pada line 4, `app:startDestination="@id/homeFragment"` menentukan fragment awal saat aplikasi dibuka.
- Pada line 6–10, terdapat deklarasi fragment pertama (HomeFragment):
 - `android:id="@+id/homeFragment"` memberikan ID ke fragment.
 - `android:name="com.example.modul5.ui.home.HomeFragment"` menunjuk ke class fragment.
 - `android:label="Home"` menampilkan label pada toolbar.
- Pada line 8–9, terdapat elemen `<action>` sebagai navigasi ke detail fragment:
 - `android:id="@+id/action_homeFragment_to_detailFragment"` memberi ID pada aksi.

- `app:destination="@id/detailFragment"` menunjukkan arah navigasi ke fragment tujuan.
- Pada line 12–17, terdapat fragment `detailFragment` untuk menampilkan detail konstelasi:
 - `android:id="@+id/detailFragment"` memberikan ID ke fragment.
 - `android:name="com.example.modul5.ui.detail.DetailFragment"` menunjuk ke class fragment.
 - `android:label="Detail"` menampilkan label "Detail".
- Pada line 15–16, terdapat elemen `<argument>` untuk menerima data objek konstelasi dari home:
 - `android:name="constellation"` nama parameter yang dikirim.
 - `app:argType="com.example.modul5.data.local.ConstellationEntity"` menunjukkan tipe data dari argumen yang diterima.
- Pada line 19, penutup elemen `<navigation>`.

23. `res/values/themes/themes.xml`

- Pada line 1, terdapat deklarasi XML dan namespace `tools` yang digunakan untuk tooling di Android Studio.
- Pada line 2–10, terdapat deklarasi style utama `Theme.MODUL5` yang mewarisi `Theme.MaterialComponents.DayNight.NoActionBar`, artinya tema ini mendukung mode gelap dan tidak menggunakan action bar default.
- Pada line 3, `colorPrimary` di-set ke `@color/purple_500` sebagai warna utama aplikasi.
- Pada line 4, `colorPrimaryVariant` di-set ke `@color/purple_700` untuk varian warna utama.
- Pada line 5, `colorOnPrimary` di-set ke `@android:color/white` agar teks/icon di atas warna utama tetap terbaca.
- Pada line 6, `colorSecondary` di-set ke `@color/teal_200` sebagai warna sekunder aplikasi.

- Pada line 7, `colorSecondaryVariant` di-set ke `@color/teal_700` sebagai varian warna sekunder.
- Pada line 8, `colorOnSecondary` di-set ke `@android:color/white` agar teks/icon di atas warna sekunder tetap terbaca.
- Pada line 10, penutup elemen `<style>` dan `<resources>`.

24. `res/values/themes/themes.xml (night)`

- Pada line 1, terdapat deklarasi XML dan namespace `tools` yang digunakan untuk tooling di Android Studio.
- Pada line 2–10, terdapat deklarasi style utama `Theme.MODUL5` yang mewarisi `Theme.MaterialComponents.DayNight.NoActionBar`, artinya tema ini mendukung mode gelap dan tidak menggunakan action bar default.
- Pada line 3, `colorPrimary` di-set ke `@color/purple_200` sebagai warna utama aplikasi.
- Pada line 4, `colorPrimaryVariant` di-set ke `@color/purple_700` untuk varian warna utama.
- Pada line 5, `colorOnPrimary` di-set ke `@android:color/white` agar teks/icon di atas warna utama tetap terbaca.
- Pada line 6, `colorSecondary` di-set ke `@color/teal_200` sebagai warna sekunder aplikasi.
- Pada line 7, `colorSecondaryVariant` di-set ke `@color/teal_700` sebagai varian warna sekunder.
- Pada line 8, `colorOnSecondary` di-set ke `@android:color/white` agar teks/icon di atas warna sekunder tetap terbaca.
- Pada line 10, penutup elemen `<style>` dan `<resources>`.

D. Tautan Git

[desyyn/PEMROGRAMAN-MOBILE](https://github.com/desyyn/PEMROGRAMAN-MOBILE)